

TAMP Known Limitations

Purpose

This document outlines the key limitations identified in the current TAMP implementation and highlights areas for future improvement.

1. Current Limitations

- **Frontend-only application** with no backend API or database.
- **Browser Local Storage** is used for data persistence, limiting scalability and multi-user support.
- **Role-based access** is enforced only on the client side and is not secure for production use.
- **Authentication** lacks features such as password recovery, session expiration, and token-based security.
- **Business rules** (e.g., duplicate requests and request limits) are only partially implemented.
- **Error handling** relies on browser alerts and console logs instead of user-friendly notifications.

2. Technical Limitations

- Duplicate match requests are not fully prevented.
- Some components require manual page refreshes to display updated data.
- Rating validation may incorrectly prevent valid submissions.
- Admin settings are stored but are not enforced throughout the application.
- No automated testing framework is currently implemented.
- Some route paths and UI logic remain hardcoded.

3. Known Issues

Issue	Impact
Duplicate match requests	Users may submit the same request multiple times.
Request button state not refreshed	Buttons may remain active until the page is refreshed.
Rating validation inconsistency	Valid ratings may be blocked incorrectly.
Local Storage errors	Invalid stored data may cause application failures.
Client-side search	Performance may decrease with large datasets.

4. Recommendations

- Integrate a backend API and database.
- Implement secure authentication and role-based authorization.
- Replace browser alerts with modern confirmation dialogs and notifications.
- Add validation to prevent duplicate records.
- Introduce automated testing using tools such as Jest and Playwright.
- Improve error handling and state management.
- Enforce administrator settings across the application.

Summary

TAMP successfully demonstrates a modular, role-based logistics platform using modern frontend technologies. However, it remains a prototype that relies on browser storage and client-side logic. Future development should focus on backend integration, stronger security, improved validation, automated testing, and enhanced user experience to make the application production-ready.